DataIngest Platform

Product Requirements Document

Field	Value
Document version	v1.0 — Initial Release
Status	Draft — Ready for AI Agent Implementation
Author	Amar Kumar Singh, Principal Solution Consultant
Date	21 March 2026
Target audience	AI Agent / Development Team
Classification	Confidential — Internal Use Only

1. Executive Summary

DataIngest is a configurable, UI-driven data ingestion and management platform designed to enable two distinct personas to interact with structured data pipelines without writing code. It solves the enterprise problem of ad-hoc, error-prone bulk data uploads by providing a governed, template-driven framework where Administrators define the rules once and Users execute against them reliably.

Core Value Proposition

Admins define the contract (template + mapping + validation + dedup rules) once. Users simply fill a structured Excel sheet and upload it. The platform handles validation, error reporting, deduplication, and writing to the correct database tables automatically — with zero code changes required when schemas evolve.

1.1 Problem Statement

Organisations managing large product catalogs, vendor registries, employee records, and asset inventories typically rely on manual import processes involving IT involvement, custom scripts, or error-prone copy-paste workflows. This results in:

- High error rates due to inconsistent data formats across teams
- Dependency on technical staff for every bulk import operation
- No audit trail or visibility into what data was imported, by whom, and when
- No enforcement of business rules at the point of data entry
- Inability to handle duplicates consistently across import jobs

1.2 Solution Overview

DataIngest provides a two-sided platform:

- Administrators configure reusable import templates by uploading an Excel schema, selecting the target database table, mapping source columns to target fields, defining duplicate handling strategy, and setting field-level validation rules. Templates are versioned and published to a catalogue.
- Users (data custodians) browse published templates, download a blank pre-formatted sheet, populate it with their data, upload it, review inline validation results, and submit. Records are written directly to the mapped database table with full traceability.

2. User Personas

2.1 Persona 1 — Administrator

Attribute	Detail
Role title	System Administrator / Data Architect
Primary goal	Define import contracts that non-technical users can execute safely
Technical level	High — understands DB schema, field types, validation patterns
Key tasks	Create templates, define mappings, set validation rules, publish, monitor
Pain points	Users submitting malformed data; no control over what lands in the DB
Success metric	Zero data quality incidents from user-submitted imports

2.2 Persona 2 — User / Data Custodian

Attribute	Detail
Role title	Business User / Data Custodian / Operations Staff
Primary goal	Upload structured data in bulk without relying on IT
Technical level	Low to medium — comfortable with Excel, not with databases
Key tasks	Browse templates, download blank sheet, fill and upload, review errors, submit
Pain points	Not knowing why an upload failed; having to contact IT for every import
Success metric	Successfully submitting a clean file on first or second attempt

3. Functional Requirements

This section defines all functional capabilities of the platform, organised by module and persona.

3.1 Admin — Template Creation Wizard

The primary Admin workflow is a multi-step wizard modal. Each step must be completed in sequence before the template can be published. Steps are accessible from the sidebar and can be navigated back once completed.

Step 1 — Upload Template File

- Admin provides an Import Job Name (required, max 120 characters)
- Admin provides an optional Import Job Description
- Admin uploads the template file via drag-and-drop or file browser
- Supported file types: .xlsx and .zip (containing xlsx)
- Maximum file size: configurable, default 50 MB
- On upload, the system parses the file and detects all column headers
- Detected column headers become the Source Fields available in Step 2
- System displays count of detected columns with a confirmation message
- Upload failure (corrupt file, wrong format) shows a clear inline error message

Step 2 — Data Mapping

This step has three sub-tabs: Data Mapping, Duplicate Handling, and Options. These tabs are accessible within Step 2 only.

Data Mapping sub-tab:

- Auto Mapping button: system attempts to match source column names to target field names automatically based on string similarity
- Definition section: Admin selects whether this template maps to a Record or an Association (radio button)
- Target Record selector: searchable dropdown populated with all available DB tables/records registered in the system
- Table list shows records grouped by schema/namespace (e.g., Global group) with a search filter
- Selected target record is confirmed with a visual indicator
- Mapping section: each row maps one Source Field (from uploaded file) to one Target Field (from selected record)
- Source Field column shows a dropdown of all detected columns from the uploaded file
- Target Field column shows a dropdown of all fields available on the selected target record
- + Add Mapping button adds a new empty mapping row
- Each mapping row has a delete (x) control
- Unmapped source fields are allowed (they will be ignored during ingestion)
- A target field can only be mapped once (duplicate target field mapping shows a warning)

Step 3 — Duplicate Handling

Admin selects one strategy from the following mutually exclusive options (radio selection):

Strategy	Behaviour
Generate new ID for all records	Every imported record is treated as new and assigned a fresh system ID. No duplicate checking is performed.
Generate new ID for duplicate records	Non-duplicates get new IDs normally. Records matching existing data also get new IDs, creating copies.
Reject duplicate records	Records matching existing data are skipped entirely and logged in the submission error report. Non-duplicates are processed normally.
Replace existing records	Records matching existing data completely overwrite the existing record. All fields are replaced with import values.
Update existing records with new data	Only the fields present in the import file are updated on matching records. Unmapped or empty fields retain their existing values.

Duplicate matching key must be specified: Admin selects which mapped field(s) are used to identify a duplicate (e.g., SKU Code, Employee ID). At least one key field is required if strategy is not "Generate new ID for all records".

Step 4 — Validation Rules & Options

Field-level Validation Rules:

- Admin can add one or more validation rules per source field
- Each rule row contains: Field (dropdown), Rule Type (dropdown), Rule Value (text input), On Error action (dropdown)
- Rule Types supported:
  - Required — field must not be empty
  - Max Length — string length must not exceed N characters
  - Min Value / Max Value — numeric range constraints
  - Decimal Places — maximum number of decimal places for numeric fields
  - Regex Pattern — value must match the provided regular expression
  - Unique in File — no duplicate values for this field within the uploaded file
  - Allowed Values (Enum) — value must be one of a defined list
- On Error action per rule: Skip Row | Warn (allow but flag) | Reject All (abort entire import)
- + Add Rule button inserts a new empty rule row; rows can be deleted individually

Ingestion Options (toggles, all default ON):

- Trim whitespace from all fields
- Normalise date formats to ISO 8601 (YYYY-MM-DD)
- Standardise currency (remove symbols, normalise decimal separators)
- Stop import on first critical error

Publishing

- A Publish button is shown at the end of Step 4 after all steps are complete
- Publishing changes template status from Draft to Live/Published
- Published templates immediately appear in the User template catalogue
- Admin can Unpublish a template to remove it from the user catalogue without deleting it
- Editing a published template creates a new Draft version; existing submissions reference the version used at submission time
- Admin can download a blank pre-formatted XLSX of any published template

3.2 Admin — Template Management

- Template list view shows all templates (draft and published) with name, category, column count, submission count, and status badge
- Admin can Edit, Duplicate, Download, or Unpublish any template from the list
- Template categories are configurable (default: Commerce, HR, IT, Procurement, Finance)
- Template versioning: each publish action increments version number

3.3 Admin — Submission Monitoring

- Admin can view all submissions across all templates in a searchable, filterable table
- Columns: Submission ID, Template Name, Submitted By, Total Rows, Error Count, Status, Submission Date
- Status values: Success (all rows written) | Partial (some rows written, some skipped) | Failed (no rows written)
- Admin can filter by template, date range, status, and submitted-by user
- Admin can click into any submission to view the full error report
- Admin can export the submission log as CSV

3.4 User — Template Catalogue

- Users see only Published templates in the catalogue
- Templates are displayed as cards with: icon, name, category, column count, and description
- Search by template name and filter by category
- Download blank template: generates a pre-formatted XLSX with locked headers, column data-type hints in row 2, and dropdown cells for Enum fields
- Template detail view shows all column names, types, and any admin-defined hints

3.5 User — Upload Wizard (3 Steps)

Step 1 — Choose Template

- User browses and selects one published template from the catalogue
- Selected template is visually highlighted with a checkmark
- Download blank template button available at this step
- Next button is disabled until a template is selected

Step 2 — Upload & Validate

- File upload zone (drag-and-drop or browse)
- Accepted formats: .xlsx, .csv matching the selected template format
- On upload, system runs all validation rules defined by Admin for that template
- Validation summary shows: Valid rows count, Error count, Total rows count
- Error table shows each failing row with: Row Number, Column Name, Error Message, and the Raw Value that failed
- User can download a full error report as CSV
- User can attempt to fix errors inline within the app before resubmitting
- If errors exist, user may choose to proceed with valid rows only (skipping errored rows) or cancel and fix the file
- Next button only enabled when at least one valid row exists

Step 3 — Confirm & Submit

- Summary card showing: Template name, filename, rows to submit (valid), rows to skip (errored)
- Optional submission note field (free text, stored with submission record)
- Preview table showing first 3 valid rows as a final confirmation
- Submit button triggers the ingestion job; records are written to the mapped DB table
- On success: confirmation screen with submission ID, total rows written, timestamp
- On failure: error screen with reason and option to retry

3.6 User — Submission History

- Users can view all their own past submissions
- Columns: Submission ID, Template, Total Rows, Valid Rows, Error Rows, Status, Submitted Date
- Click into any submission to see the full error report for that submission
- Download error report for any past submission

4. Core Data Model

The following entities form the backbone of the DataIngest platform. All entities should be persisted in a relational database. The ingestion engine reads from these entities at runtime to execute the correct mapping and validation logic.

Entity	Key Fields	Purpose
TEMPLATE	id, name, description, category, status, version, created_by, published_at	Defines an import job. Status: draft | published | deprecated.
TEMPLATE_COLUMN	id, template_id, column_name, display_label, data_type, is_required, sort_order, target_table, target_field	Each row is one source-to-target field mapping for a template.
VALIDATION_RULE	id, column_id, rule_type, rule_value, on_error, error_message	One or more rules attached to a template column. Evaluated at ingestion time.
DUPLICATE_CONFIG	id, template_id, strategy, key_fields (JSON array)	Deduplication strategy and the key field(s) used for matching.
SUBMISSION	id, template_id, template_version, submitted_by, status, total_rows, success_rows, error_rows, note, submitted_at	One record per upload job. Links user, template version, and outcome.
SUBMISSION_ERROR	id, submission_id, row_number, column_name, rule_type, raw_value, error_message	Row-level error detail for each failed validation during a submission.

5. Non-Functional Requirements

Category	Requirement
Performance	File validation for up to 10,000 rows must complete within 5 seconds
Performance	UI interactions (page navigation, modal open) must respond within 200ms
Scalability	System must support concurrent ingestion jobs from multiple users without data collision
File Size	Maximum supported file size: 50 MB (configurable by Admin)
Security	Role-based access: Admin and User roles with distinct permissions
Security	Users can only view their own submission history, not other users'
Security	All file uploads scanned for malicious content before processing
Audit	All submissions logged with user identity, timestamp, template version, and outcome
Data Integrity	Template publish is immutable — published version cannot be edited, only deprecated
Accessibility	UI must meet WCAG 2.1 AA standards
Browser Support	Latest two versions of Chrome, Firefox, Safari, and Edge
Uptime	99.5% availability SLA for production environment

6. UI / UX Requirements

6.1 Admin Interface

- Multi-step wizard modal for template creation (not a separate page)
- Wizard steps shown in a left sidebar with status indicators: incomplete | active | complete
- Step 2 wizard panel contains three sub-tabs: Data Mapping | Duplicate Handling | Options
- Auto Mapping button in Data Mapping tab — one-click intelligent field matching
- Searchable table/record selector with grouped list (by namespace/schema)
- Mapping rows: source dropdown → arrow → target dropdown → delete button
- Duplicate handling: radio card selection (one option per card with label and description)
- Validation rules: inline table with Add/Delete row controls per field
- Publish button shown prominently at end of Step 4 with a "ready to publish" confirmation banner
- Admin dashboard shows summary stats: active templates, total submissions, success rate, active users

6.2 User Interface

- Template catalogue with card grid layout, search, and category filter
- Three-step upload wizard with horizontal step progress indicator
- Upload zone with drag-and-drop support and click-to-browse fallback
- Validation results: summary bar (valid / errors / total) + inline error table with row numbers
- Confirm & Submit screen shows submission summary with data preview (first 3 rows)

6.3 General UX Principles

- Light mode (white surface) theme as the default
- All destructive or irreversible actions require explicit confirmation
- Loading states shown for file upload, validation, and submission operations
- Toast notifications for success and error events
- Empty states with clear call-to-action messaging
- All form fields with validation show inline error messages (not alert boxes)

7. User Stories & Acceptance Criteria

7.1 Admin Stories

US-A01 — Create Import Template

As an Admin, I want to create a new import template by uploading an Excel file, so that I can define a reusable import job without writing code.

Acceptance Criteria:

- Admin can upload .xlsx or .zip file in Step 1 of the wizard
- System detects and displays column headers from the uploaded file
- Import job name is required before proceeding to Step 2
- File upload failure shows a clear error message with the reason

US-A02 — Map Columns to Target Fields

As an Admin, I want to map each source column from my Excel file to the correct target field in the database, so that ingested data lands in the right place.

Acceptance Criteria:

- Admin can select a target Record or Association from a searchable list
- Auto Mapping button suggests mappings based on column name similarity
- Admin can add, edit, and delete individual mapping rows
- Mapping the same target field twice shows a duplicate warning
- At least one mapping row is required before proceeding to Step 3

US-A03 — Configure Duplicate Handling

As an Admin, I want to define how the system handles duplicate records, so that data integrity is maintained according to business rules.

Acceptance Criteria:

- Admin can select exactly one duplicate handling strategy
- All five strategies are available as selectable options with descriptions
- For non-"generate all" strategies, admin must specify key field(s) for matching
- Selected strategy is saved with the template and enforced at ingestion time

US-A04 — Define Field Validation Rules

As an Admin, I want to set field-level validation rules on template columns, so that only clean, compliant data is accepted from users.

Acceptance Criteria:

- Admin can add multiple rules per field
- All rule types (required, max length, regex, min/max value, unique, enum) are available
- Each rule has a configurable "on error" action: skip row, warn, or reject all
- Custom error messages can be defined per rule (shown to user on validation failure)

US-A05 — Publish Template

As an Admin, I want to publish a completed template so that users can find and use it.

Acceptance Criteria:

- Publish button is only active when all four wizard steps are complete
- Published template immediately appears in the User catalogue
- Template status changes from Draft to Live with a version number assigned
- Admin can unpublish a template to hide it from the user catalogue

7.2 User Stories

US-U01 — Browse and Download Template

As a User, I want to browse available templates and download a blank Excel sheet, so that I know exactly what format to fill my data in.

Acceptance Criteria:

- User sees only Published templates in the catalogue
- Download button generates a pre-formatted XLSX with locked headers
- Enum fields in downloaded template use Excel data validation dropdowns
- Row 2 of downloaded template shows data type hints (e.g., "Text, max 120 chars")

US-U02 — Upload and Validate File

As a User, I want to upload my filled Excel file and see which rows have errors, so that I can fix issues before submitting.

Acceptance Criteria:

- User can drag-and-drop or browse to upload a file
- Validation runs automatically on upload and completes within 5 seconds for files up to 10,000 rows
- Validation summary shows total rows, valid count, and error count
- Each error shows the row number, column name, and the specific rule that failed
- User can download a CSV error report
- User cannot proceed if zero valid rows exist

US-U03 — Confirm and Submit

As a User, I want to review a summary of my submission and confirm before records are written to the database.

Acceptance Criteria:

- Confirm screen shows template name, file name, valid row count, and skipped row count
- First 3 valid rows shown as a data preview table
- User can add an optional submission note
- Submit button triggers ingestion; a loading state is shown during processing
- On success: confirmation with submission ID and rows written count
- On failure: error screen with reason and retry option

US-U04 — View Submission History

As a User, I want to see all my past submissions and their outcomes, so that I have a clear audit trail of my data imports.

Acceptance Criteria:

- User can see only their own submissions (not other users')
- Each row shows: submission ID, template, row counts, status, and date
- User can click into any submission to view the error detail
- User can download the error report for any past submission

8. Feature Priority & MVP Scope

The following table defines the MVP scope for the initial AI Agent implementation sprint. Features marked P1 are required for MVP. P2 and P3 follow in subsequent iterations.

Feature / Requirement	Priority	Notes
Admin: Template creation wizard (all 4 steps)	P1 — MVP	Core admin capability
Admin: Upload Excel file to define columns	P1 — MVP	Core admin capability
Admin: Searchable target table/record selector	P1 — MVP	Core admin capability
Admin: Source-to-target field mapping (manual)	P1 — MVP	Core admin capability
Admin: Duplicate handling strategy configuration	P1 — MVP	Core admin capability
Admin: Field-level validation rule builder	P1 — MVP	Core admin capability
Admin: Publish / unpublish template	P1 — MVP	Core admin capability
Admin: Template management list (edit, duplicate, download)	P1 — MVP	Core admin capability
Admin: Submission monitoring dashboard	P1 — MVP	Core admin capability
User: Browse published template catalogue	P1 — MVP	Core user capability
User: Download pre-formatted blank template XLSX	P1 — MVP	Core user capability
User: Upload file and run validation	P1 — MVP	Core user capability
User: Inline error report with row-level detail	P1 — MVP	Core user capability
User: Confirm & submit with data preview	P1 — MVP	Core user capability
User: Submission history table	P1 — MVP	Core user capability
Admin: Auto mapping (AI-assisted field matching)	P2	Quality of life
User: Inline cell editing before submission	P2	Quality of life
Template versioning with history	P2	Governance
Export submission log as CSV	P2	Reporting
Email notifications on submission	P3	Nice to have
REST API / webhook export integration	P3	Nice to have
Column-level PII masking	P3	Compliance

9. Technical Architecture Guidance

The following guidance is provided to the AI Agent implementing this platform. Architecture decisions should optimise for speed of delivery while maintaining extensibility.

9.1 Recommended Stack

Layer	Recommendation	Rationale
Frontend	React + TypeScript	Component-based, typed, widely supported by AI code agents
Styling	Tailwind CSS	Utility-first, rapid development, consistent design tokens
State management	Zustand or React Context	Lightweight, sufficient for this app complexity
File parsing (client)	SheetJS (xlsx)	Parse xlsx/csv in-browser without server round-trip
Backend API	Node.js + Express or FastAPI	Lightweight REST API for template CRUD and ingestion jobs
Database	PostgreSQL	Relational schema fits the entity model; JSONB for dynamic configs
File storage	S3-compatible (AWS S3 / MinIO)	Store uploaded template files and submission artefacts
Auth	JWT + Role-based middleware	Admin and User roles enforced at API layer

9.2 Key Implementation Notes

- The ingestion engine must read TEMPLATE_COLUMN and VALIDATION_RULE records at runtime — it must not hardcode any business logic per template
- Validation should run client-side first (fast feedback) and be re-validated server-side before write (security)
- Template file upload in the Admin wizard should be parsed immediately on upload to extract column headers — this drives the source field dropdowns in Step 2
- The downloaded blank template for users must be generated dynamically from the template configuration, not stored as a static file
- Duplicate detection must use the key_fields from DUPLICATE_CONFIG and query the target table before writing
- Each submission must write to SUBMISSION and SUBMISSION_ERROR tables transactionally — partial writes should not occur
- Template version number must be stamped on each SUBMISSION record so error reports always reference the correct version

10. Open Questions & Assumptions

10.1 Open Questions

#	Question	Owner	Status
OQ-1	Will the platform connect to live DB tables at runtime, or will target tables be pre-registered by admin?	Product	Open
OQ-2	Should users be able to submit partial files (valid rows only) by default, or require admin to configure this per template?	Product	Open
OQ-3	What authentication provider will be used? (SSO, LDAP, local auth?)	Engineering	Open
OQ-4	Is there a requirement for multi-tenancy (different organisations sharing the platform)?	Product	Open
OQ-5	Should template categories be hardcoded or dynamically managed by Admin?	Product	Assumed dynamic

10.2 Assumptions

- The platform is a single-tenant application (one organisation) for the initial release
- Target tables are pre-registered or discoverable via a DB connection configured by an infrastructure admin
- File storage (uploaded templates and submission artefacts) uses an S3-compatible service
- The AI Agent has access to the UI mockup (dataingest-v2.html) as a visual reference for all screen designs
- The data model described in Section 4 is the starting schema; the AI Agent may propose adjustments with justification

Appendix A — Glossary

Term	Definition
Template	A configured import job definition created by Admin, comprising column mappings, validation rules, and duplicate strategy
Source Field	A column header detected from the Admin-uploaded Excel template file
Target Field	A field in the destination database table/record that a source field is mapped to
Submission	One completed upload event by a User against a specific template version
Ingestion Engine	The backend service that reads template configuration and writes validated rows to the target table
Duplicate Key	The field(s) used to identify whether an imported record already exists in the target table
Data Custodian	A business user responsible for uploading and maintaining data in a particular domain
Draft	A template that has not yet been published; not visible to Users
Published / Live	A template that has been published by Admin and is available in the User catalogue

Appendix B — Reference Artefacts

The following artefacts were produced during the discovery and design phase and should be used as reference by the implementing AI Agent:

- UI Mockup: dataingest-v2.html — fully interactive light-mode prototype of both Admin and User portals including the 4-step Admin wizard and 3-step User upload wizard
- Admin wizard PDF reference: Data_Management_-_Admin.pdf — original wireframe screenshots provided by the product owner
- Conversation transcript: full requirement discovery session including capability map, data model ERD, and persona journey diagrams

End of Document